Algorithmes et Systèmes Distribués

Corrections détaillées des TD et TP - M1 Informatique 2026/2027 (E. Fabiani)

Ce document corrige, étape par étape, les **TD2 à TD5** puis les **TP2, TP3 (parties 1 et 2) et TP4**. Le chronogramme fourni est traité dans le TP2 (question 2.4).

Conventions communes

- Langage : Go. Les canaux sont synchrones (non bufférisés) : un envoi bloque jusqu'à ce qu'un récepteur soit prêt.
- Modèle synchrone : à chaque tour, un nœud envoie un message sur chaque canal de sortie et en reçoit un sur chaque canal d'entrée (éventuellement `null`), puis change d'état. Tous les nœuds doivent faire le même nombre de tours, sinon un voisin reste bloqué.
- La fonction `communication(in, messIn, out, messOut)` (TP3 partie 1) réalise l'échange d'un tour. Les TD3 à TD5 l'utilisent telle quelle.
- Les exemples chiffrés (TP3, TP4) ont été vérifiés par simulation sur une topologie de 7 nœuds donnée dans le TP3 partie 1. Votre propre topologie donnera d'autres valeurs : seuls les raisonnements sont à reprendre.
- Le code est écrit pour être lu et recopié, mais je n'ai pas pu le compiler ici (pas de compilateur Go disponible) : un test `go run` reste à faire de votre côté.

Sommaire

Partie A - TD	TD2 Synchronisations centralisées | TD3 BFS synchrone | TD4 Algorithmes sur arbre BFS | TD5 Commutation
Partie B - TP	TP2 Élection sur anneau (+ chronogramme) | TP3.1 Topologie et communications | TP3.2 Inondation | TP4 RIP

PARTIE A - Travaux dirigés

TD2 - Synchronisations centralisées

Méthode imposée : (1) schéma du système, (2) ossature (`main` : canaux + lancement des processus), (3) comportement des processus, en boucle infinie. Dans tous les exercices, le `main` se termine par `select {}` : il se bloque sans consommer de CPU pendant que les goroutines tournent (on arrête avec Ctrl+C). Rappel : un canal synchrone (non bufférisé) bloque l'émetteur tant qu'un récepteur n'est pas prêt, et inversement ; c'est ce rendez-vous qui sert de synchronisation.

1.1 Élection centralisée

Étape 1 - Schéma

```
 utilisateur 0 --versCtrl[0]-->  +-------------+ --depuisCtrl[0]--> utilisateur 0
 utilisateur 1 --versCtrl[1]-->  | contrôleur  | --depuisCtrl[1]--> utilisateur 1
 utilisateur 2 --versCtrl[2]-->  +-------------+ --depuisCtrl[2]--> utilisateur 2
   (chan int : valeur d'élection)                (chan int : valeur de l'élu)
```

Deux tableaux de 3 canaux : un par sens. Chaque utilisateur envoie sa valeur, puis attend la réponse.

Étapes 2 et 3 - Code

```
package main

import ("fmt"; "time")

const nbUsers = 3

func controleur(in []chan int, out []chan int) {
    for {
        max := -1                       // valeurs d'élection >= 0
        for i := range in {
            v := <-in[i]                // 1) collecte des 3 valeurs
            if v > max { max = v }
        }
        for i := range out {
            out[i] <- max               // 2) diffusion de la valeur de l'élu
        }
    }
}

func utilisateur(id, val int, versCtrl, depuisCtrl chan int) {
    for {
        versCtrl <- val                 // je propose ma valeur (bloquant)
        elu := <-depuisCtrl             // j'attends la valeur élue
        if elu == val {
            fmt.Println("Processus", id, ": je suis élu")
        } else {
            fmt.Println("Processus", id, ": non élu, l'élu a", elu)
        }
        time.Sleep(time.Second)         // seulement pour lisibilité de la trace
    }
}

func main() {
    var versCtrl, depuisCtrl [nbUsers]chan int
    for i := range versCtrl {
        versCtrl[i] = make(chan int)
        depuisCtrl[i] = make(chan int)
    }
    valeurs := [nbUsers]int{12, 5, 30}  // uniques, fixées par paramètre
    go controleur(versCtrl[:], depuisCtrl[:])
    for i := 0; i < nbUsers; i++ {
        go utilisateur(i, valeurs[i], versCtrl[i], depuisCtrl[i])
    }
    select {}
}
```

Pourquoi ça ne bloque pas : le contrôleur lit les canaux dans l'ordre 0, 1, 2 ; les utilisateurs ne font qu'envoyer puis attendre, aucun n'a besoin d'être lu dans un ordre précis. Ensuite le contrôleur envoie à chacun, et tous sont déjà en attente de réception.

1.2 Exclusion mutuelle centralisée

Question 1 - un seul tableau de canaux, avec select

Schéma : `tabCan[i]` relie l'utilisateur i au contrôleur (sens utilisateur vers contrôleur uniquement). Protocole : (a) demande = envoi sur `tabCan[i]` ; l'utilisateur est débloqué quand le contrôleur accepte, ce qui vaut autorisation ; (b) le contrôleur attend alors la libération, un second envoi sur le même `tabCan[i]`. Pendant ce temps il n'écoute plus personne d'autre : les autres demandeurs restent bloqués sur leur envoi, d'où l'exclusion mutuelle.

```
func controleur(tabCan []chan int) {
    for {
        select {
        case <-tabCan[0]:
            fmt.Println("Processus 0 en section critique")
            <-tabCan[0]                  // attente de la libération de 0
        case <-tabCan[1]:
            fmt.Println("Processus 1 en section critique")
            <-tabCan[1]
        case <-tabCan[2]:
            fmt.Println("Processus 2 en section critique")
            <-tabCan[2]
        }
    }
}

func utilisateur(id int, can chan int) {
    for {
        // section non critique
        can <- id                        // demande (bloque jusqu'à acceptation)
        // section critique
        time.Sleep(500 * time.Millisecond)
        can <- id                        // libération
    }
}

func main() {
    var tabCan [3]chan int
    for i := range tabCan { tabCan[i] = make(chan int) }
    go controleur(tabCan[:])
    for i := 0; i < 3; i++ { go utilisateur(i, tabCan[i]) }
    select {}
}
```

La direction a-t-elle une importance ? Oui. L'initiative doit venir de l'utilisateur : c'est lui qui veut entrer, donc c'est lui qui émet vers le contrôleur (sens utilisateur vers contrôleur). Si le contrôleur émettait (`case tabCan[i] <- ...` dans le select), il donnerait la permission à un processus qui ne l'a peut-être pas demandée, et ne saurait pas qui veut entrer. Ici la réception par le contrôleur est la permission.

Question 2 - un canal partagé, sans select

```
func controleur(req, lib chan int) {
    for {
        id := <-req                      // n'importe quel demandeur
        fmt.Println("Processus", id, "en section critique")
        <-lib                            // attend la libération
    }
}
func utilisateur(id int, req, lib chan int) {
    for {
        req <- id          // demande
        time.Sleep(500 * time.Millisecond)   // section critique
        lib <- id          // libération
    }
}
// main : req := make(chan int); lib := make(chan int); go controleur(req, lib) ...
```

Est-ce possible avec un seul canal ? Non. Si le même canal sert aux demandes et aux libérations, le contrôleur, qui attend la libération, peut recevoir à la place la demande d'un autre processus : il ne peut ni la refuser ni la distinguer d'une libération, et l'émetteur de cette demande est débloqué, donc entrerait en section critique en même temps. Il faut donc deux canaux (`req` et `lib`), chacun étant partagé par plusieurs producteurs.

1.3 Cohorte centralisée (au plus nbMax processus)

Le contrôleur compte les processus présents (`nb`). Il doit accepter une libération à tout moment, mais une demande seulement si `nb < nbMax`. Astuce Go : dans un `select`, un canal `nil` n'est jamais prêt ; on désactive donc le cas des demandes en mettant le canal local à `nil` quand la cohorte est pleine.

```
const nbMax = 2

func controleur(req, lib chan int) {
    nb := 0
    for {
        var r chan int            // nil : ce cas du select est bloqué
        if nb < nbMax { r = req } // sinon les demandes ne sont plus acceptées
        select {
        case id := <-r:
            nb++
            fmt.Println("Processus", id, "entre, présents :", nb)
        case id := <-lib:
            nb--
            fmt.Println("Processus", id, "sort, présents :", nb)
        }
    }
}
// utilisateur : identique à 1.2 question 2 ; avec nbMax = 1 on retrouve l'exclusion mutuelle.
```

TD3 - Algorithme BFS dans le modèle synchrone

Rappel du modèle synchrone : à chaque tour, chaque nœud envoie un message sur **chaque** canal de sortie et en reçoit un sur **chaque** canal d'entrée. D'où le message `null` : sans lui, un nœud sans information à transmettre bloquerait ses voisins. La fonction `communication(in, messIn, out, messOut)` (écrite dans le TP3) fait cet échange.

1. BFS dans un réseau unidirectionnel (version 1 : parent seul)

Étape 1 - Entête et variables locales

```
const search = 1
const null = 0
const diametre = 5

func node(in []chan int, out []chan int, racine bool) {
    var messIn []int      // un message par canal d'entrée
    var messOut []int     // un message par canal de sortie
    var visite bool       // vrai dès la première réception de search
    var parent int        // indice du canal d'entrée vers le parent

    messIn = make([]int, len(in))      // taille = nb de canaux (slice)
    messOut = make([]int, len(out))
```

Les tableaux sont des slices dimensionnées sur les tableaux de canaux : le même code fonctionne pour tout nœud, quel que soit son nombre de voisins.

Étape 2 - Initialisation

```
    visite = false
    parent = -1                        // pas de parent connu
    for i := range messOut { messOut[i] = null }
    if racine {
        visite = true                  // la racine est marquée d'office
        for i := range messOut { messOut[i] = search }   // elle lance le parcours
    }
```

Étapes 3 et 4 - Boucle synchrone et changement d'état

```
    for tour := 0; tour < diametre; tour++ {
        // 3. communications (envoi de messOut, réception dans messIn)
        communication(in, messIn, out, messOut)

        // 4. changement d'état : par défaut on enverra null au prochain tour
        for i := range messOut { messOut[i] = null }

        if !visite {
            for i := range messIn {
                if messIn[i] == search {
                    visite = true
                    parent = i         // canal de première réception
                    break              // si plusieurs search arrivent, on en prend un
                }
            }
            if visite {                // on propage la recherche à tous les voisins
                for j := range messOut { messOut[j] = search }
            }
        }
    }
}
```

Pourquoi `diametre` tours suffisent : la racine émet au tour 1 ; un nœud à distance d reçoit `search` au tour d. La profondeur maximale est le diamètre, donc `diametre` tours couvrent tous les nœuds. Un nœud déjà visité ignore les `search` suivants : c'est ce marquage qui garantit que le parent est bien situé sur un plus court chemin.

2. BFS dans un réseau bidirectionnel (version 2 : parent et fils)

Changements : (1) variables `nbfils` et `tabfils` ; (2) message `fils = 2` ; (3) un nœud nouvellement marqué envoie `search` à tous ses voisins **sauf** son parent, auquel il envoie une seule fois `fils` ; (4) un nœud scrute aussi `fils` à tout moment ; (5) il faut un tour de plus : le dernier `fils` part au tour diametre+1 depuis un nœud à profondeur diametre.

```
const fils = 2
```

```
func node(in []chan int, out []chan int, racine bool) {
    var messIn, messOut []int
    var visite bool
    var parent, nbfils int
    var tabfils []int                       // indices des canaux vers les fils (in = out)
    messIn = make([]int, len(in))
    messOut = make([]int, len(out))

    visite, parent, nbfils = false, -1, 0
    tabfils = make([]int, 0)
    for i := range messOut { messOut[i] = null }
    if racine {
        visite = true
        for i := range messOut { messOut[i] = search }
    }

    for tour := 0; tour < diametre+1; tour++ {
        communication(in, messIn, out, messOut)
        for i := range messOut { messOut[i] = null }

        for i := range messIn {
            switch messIn[i] {
            case fils:                      // un voisin se déclare mon fils
                nbfils++
                tabfils = append(tabfils, i)
            case search:
                if !visite {                // première réception : i devient mon parent
                    visite = true
                    parent = i
                    for j := range messOut {
                        if j != parent { messOut[j] = search }
                    }
                    messOut[parent] = fils // une seule fois, au tour suivant
                }
            }
        }
    }
}
```

`fils` ne peut arriver que sur un nœud déjà visité (seul un parent le reçoit), donc le `switch` sur `fils` n'a pas besoin du test `!visite`. Après la boucle, chaque nœud connaît `parent`, `nbfils` et `tabfils`.

TD4 - Algorithmes basés sur un arbre BFS

Hypothèses : le BFS du TD3 a été exécuté, et la liaison avec un voisin utilise le même indice en entrée et en sortie (il faut pouvoir écrire dans `out[parent]`). On fixe `const null = -1` (ce qui permet d'utiliser 0 comme valeur d'algorithme, cf. 1.2) et des identifiants entiers >= 0. Le principe est un **calcul remontant** (convergecast) : chaque nœud résume ce qu'il sait de son sous-arbre et le donne à son parent.

1.1 Maximum des identifiants (résultat chez la racine)

Variante 1 : seule la variable parent est disponible

Un nœud ne connaît pas ses fils, donc il ne sait pas quand il a tout reçu. Il envoie donc son maximum courant **à chaque tour** à son parent. Aucun risque de confusion : un nœud ne reçoit des valeurs que de ceux dont il est le parent, c'est-à-dire ses fils. L'information remonte d'un niveau par tour : `diametre` tours suffisent.

```
max := id
for i := range messOut { messOut[i] = null }
if !racine { messOut[parent] = max }            // les feuilles démarrent la remontée

for tour := 0; tour < diametre; tour++ {
    communication(in, messIn, out, messOut)
    for i := range messIn {
        if messIn[i] != null && messIn[i] > max { max = messIn[i] }
    }
    for i := range messOut { messOut[i] = null }
    if !racine { messOut[parent] = max }        // renvoi du max courant à chaque tour
}
// à la fin, si racine : max = maximum de tous les identifiants
```

Variante 2 : parent, tabFils et nbFils disponibles

On sait exactement combien de messages attendre : un par fils. Le nœud compte les valeurs reçues (`recus`) et n'envoie son maximum qu'**une seule fois**, quand `recus == nbFils`. Une feuille (nbFils = 0) envoie dès le début. Le trafic utile tombe à n-1 messages non nuls (un par arête de l'arbre).

```
max, recus, envoye := id, 0, false
for i := range messOut { messOut[i] = null }
if !racine && nbFils == 0 {                      // feuille : envoi immédiat
    messOut[parent] = max
    envoye = true
}
for tour := 0; tour < diametre; tour++ {
    communication(in, messIn, out, messOut)
    for i := range messOut { messOut[i] = null }
    for _, f := range tabFils {
        if messIn[f] != null {
            recus++
            if messIn[f] > max { max = messIn[f] }
        }
    }
    if !racine && !envoye && recus == nbFils {   // tous les fils ont répondu
        messOut[parent] = max
        envoye = true
    }
}
```

1.2 Distance maximale depuis la racine

Chaque nœud calcule la **hauteur de son sous-arbre** (une feuille : 0). Il transmet cette hauteur à son parent, qui ajoute 1 : `dist = max(dist, reçu + 1)`. La racine obtient ainsi la plus grande profondeur, donc la distance maximale. C'est ici que `null = -1` est indispensable : 0 est une vraie valeur (feuille), il ne doit pas jouer le rôle de null.

```
// Variante 1 (parent seul) : on remplace max par dist
dist := 0
for i := range messOut { messOut[i] = null }
if !racine { messOut[parent] = dist }
for tour := 0; tour < diametre; tour++ {
    communication(in, messIn, out, messOut)
    for i := range messIn {
        if messIn[i] != null && messIn[i]+1 > dist { dist = messIn[i] + 1 }
    }
    for i := range messOut { messOut[i] = null }
```

```
    if !racine { messOut[parent] = dist }
}

// Variante 2 (avec fils) : même schéma que 1.1 variante 2, seule la ligne de mise à jour change
//     if messIn[f] != null {
//         recus++
//         if messIn[f]+1 > dist { dist = messIn[f] + 1 }
//     }
// et on envoie dist (et non max) au parent quand recus == nbFils.
```

2.1 Communication restreinte à la topologie virtuelle BFS bidirectionnelle

Q1. Dans le cas général, un nœud n'envoie qu'à son parent et à ses fils, et ne reçoit que de son parent et de ses fils. Les autres voisins du réseau d'origine ne sont plus concernés (la racine n'a pas de parent, les feuilles pas de fils).

Q2. On construit des tableaux réduits aux seuls voisins BFS et on appelle `communication` dessus :

```
// (a) nombre de voisins dans la topologie virtuelle
nbVois := nbFils
if !racine { nbVois++ }

// (b) messages adaptés
messInBFS := make([]int, nbVois)
messOutBFS := make([]int, nbVois)

// (c) canaux utiles uniquement, (d) nouveaux indices dans la topologie BFS
inBFS := make([]chan int, nbVois)
outBFS := make([]chan int, nbVois)
tabFilsBFS := make([]int, nbFils)
parentBFS := -1
k := 0
for _, f := range tabFils {          // les fils d'abord : indices 0 .. nbFils-1
    inBFS[k] = in[f]
    outBFS[k] = out[f]
    tabFilsBFS[k] = k
    k++
}
if !racine {                         // le parent en dernier
    inBFS[k] = in[parent]
    outBFS[k] = out[parent]
    parentBFS = k
}

// phase de communication (à chaque tour) :
communication(inBFS, messInBFS, outBFS, messOutBFS)
```

Les indices (`parentBFS`, `tabFilsBFS`) sont des indices dans les nouveaux tableaux, plus dans `in`/`out`. Les deux extrémités d'un canal BFS utilisent bien le même canal physique : pas d'interblocage.

2.2 Communication restreinte à un BFS unidirectionnel

Q1. Pour un calcul global (remontée), les communications utiles vont des fils vers le parent (vers la racine).

Q2. On ne garde que : en entrée, les canaux venant des fils ; en sortie, le canal vers le parent. Le sens parent vers fils n'est utile que pour une diffusion.

Q3. Code (basé sur 2.1) :

```
nbOut := 0
if !racine { nbOut = 1 }                        // un seul canal de sortie : vers le parent
messInBFS := make([]int, nbFils)                // un message par fils
messOutBFS := make([]int, nbOut)
inBFS := make([]chan int, nbFils)
outBFS := make([]chan int, nbOut)
for k, f := range tabFils { inBFS[k] = in[f] }  // tabFilsBFS[k] = k
if !racine { outBFS[0] = out[parent] }          // parentBFS = 0 (côté sortie)
communication(inBFS, messInBFS, outBFS, messOutBFS)
```

- **Conservé :** l'appel à `communication`, les tableaux `messInBFS`/`messOutBFS`, les canaux `inBFS` vers les fils, `tabFilsBFS` (indices 0..nbFils-1) et le canal de sortie vers le parent.
- **Inutile :** les canaux de sortie vers les fils et le canal d'entrée venant du parent, le total `nbVois`, et `parentBFS` côté entrée. La racine n'a aucun canal de sortie, une feuille aucun canal d'entrée : `communication` gère sans problème les tableaux de taille 0.

2.3 Élection dans la topologie virtuelle bidirectionnelle

Phase 1 : remontée du maximum (cf. 1.1 variante 2, mais avec les tableaux BFS). Phase 2 : la racine diffuse cette valeur vers le bas ; un nœud qui reçoit son propre identifiant est élu.

```
// ---- Phase 1 : remontée (tableaux BFS de 2.1) ----
max, recus, envoye := id, 0, false
for i := range messOutBFS { messOutBFS[i] = null }
if !racine && nbFils == 0 { messOutBFS[parentBFS] = max; envoye = true }
for tour := 0; tour < diametre; tour++ {
    communication(inBFS, messInBFS, outBFS, messOutBFS)
    for i := range messOutBFS { messOutBFS[i] = null }
    for _, f := range tabFilsBFS {
        if messInBFS[f] != null {
            recus++
            if messInBFS[f] > max { max = messInBFS[f] }
        }
    }
    if !racine && !envoye && recus == nbFils {
        messOutBFS[parentBFS] = max
        envoye = true
    }
}

// ---- Phase 2 : diffusion descendante ----
leader := -1
for i := range messOutBFS { messOutBFS[i] = null }
if racine {                                      // la racine connaît le max
    leader = max
    for _, f := range tabFilsBFS { messOutBFS[f] = leader }
}
for tour := 0; tour < diametre; tour++ {
    communication(inBFS, messInBFS, outBFS, messOutBFS)
    for i := range messOutBFS { messOutBFS[i] = null }
    if !racine && messInBFS[parentBFS] != null {  // valeur reçue du parent
        leader = messInBFS[parentBFS]
        for _, f := range tabFilsBFS { messOutBFS[f] = leader }   // je la relaie
    }
}
if leader == id { fmt.Println("Noeud", id, ": élu") }
```

Les deux phases durent chacune `diametre` tours : tous les nœuds font le même nombre d'échanges, sinon un voisin resterait bloqué. Il n'y a que des messages sur les arêtes de l'arbre, donc beaucoup moins de trafic que sur le réseau complet.

TD5 - Algorithme de commutation (switch auto-apprenant)

Un commutateur reçoit une trame sur un port, **apprend** que l'émetteur est joignable par ce port, puis relaie la trame : vers le seul port connu de la destination, ou sur tous les autres ports (inondation) si elle est inconnue.

```
const null = -1
type trame struct { from, to int }                // from == null : message null
var trameNull = trame{from: null, to: null}
```

4.1 Entête, variables, initialisation, boucle

- **Q1.** Il faut connaître le nombre de machines du réseau, `nbMachines` : adresses 0 à nbMachines-1, continues, donc `tableCom` est indexée par l'adresse de la machine.
- **Q2.** Les ports sont numérotés à partir de 0 : une valeur négative, par exemple -1, ne peut jamais être un port valide. -1 signifie « machine pas encore apprise ».
- **Q3.** Toutes les cases de `tableCom` sont initialisées à -1 (les tables sont vides au départ).

Q4 à Q6 - Code

```
func commutateur(in []chan trame, out []chan trame) {
    // Q4 : table de commutation (taille statique) initialisée à -1
    var tableCom [nbMachines]int
    for i := range tableCom { tableCom[i] = -1 }

    // Q5 : messages (taille = nb de ports) alloués puis initialisés à null
    messIn := make([]trame, len(in))
    messOut := make([]trame, len(out))
    for i := range messIn { messIn[i] = trameNull }
    for i := range messOut { messOut[i] = trameNull }

    // Q6 : boucle infinie + communication synchrone
    for {
        communication(in, messIn, out, messOut)
        // ... 4.2 et 4.3 ...
    }
}
```

4.2 et 4.3 - Mise à jour de la table et relais (version sans file d'attente)

Dans le corps de la boucle, après la communication : (1) remise à `null` des sorties (**avant** de les préparer, sinon on renverrait l'ancien tour) ; (2) pour chaque trame non nulle reçue sur le port p : apprentissage `tableCom[from] = p`, puis relais selon que la destination est connue ou non.

```
        for i := range messOut { messOut[i] = trameNull }        // remise à null

        for p := range messIn {
            t := messIn[p]
            if t.from == null { continue }                       // rien reçu sur ce port

            tableCom[t.from] = p                                 // 4.2 apprentissage

            dest := tableCom[t.to]                               // 4.3 relais
            if dest == -1 {                                      // destination inconnue
                for q := range messOut {
                    if q != p { messOut[q] = t }                 // diffusion sauf port d'origine
                }
            } else if dest != p {                                // connue, et pas du même côté
                messOut[dest] = t
            }                                          // dest == p : même secteur, on ignore
        }
```

Conflits : si deux trames visent le même port de sortie dans un cycle, la dernière écrite écrase la précédente. C'est la limite que corrige la version avec files d'attente.

5.1 Files d'attente de capacité suffisante

5.1.1 Création des fifos

Il faut **une fifo par port de sortie**, soit `len(out)` files : les conflits se produisent à la sortie. Ce sont des canaux bufférisés **locaux** au commutateur (créés dans la fonction, non partagés).

```
fifos := make([]chan trame, len(out))
for i := range fifos { fifos[i] = make(chan trame, tailleFifo) }
```

5.1.2 Écriture : on empile au lieu d'écraser messOut

```
if dest == -1 {
    for q := range fifos {
        if q != p { fifos[q] <- t }       // au lieu de messOut[q] = t
    }
} else if dest != p {
    fifos[dest] <- t                      // au lieu de messOut[dest] = t
}
```

5.1.3 Lecture : une trame par port et par tour

Après la boucle d'examen, pour préparer le tour suivant, on extrait au plus une trame de chaque file (un seul message par canal et par tour). `len` sur un canal donne le nombre d'éléments en attente.

```
for q := range fifos {
    if len(fifos[q]) > 0 {
        messOut[q] = <-fifos[q]
    }
}
```

5.1.4 Problème des messages null

- **Q1 - réinitialisation systématique :** elle est correcte seulement si elle précède l'extraction des files. Placée après (ou en fin de tour), elle écrase la trame qui vient de sortir de la file : la trame est perdue, alors qu'elle n'a jamais été envoyée.
- **Q2 - sans réinitialisation :** quand la file est vide, `messOut[q]` conserve la dernière trame envoyée, qui est donc **renvoyée à chaque tour** (duplication infinie).
- **Q3 - information nécessaire :** savoir si la file du port est vide, ce que donne `len(fifos[q])`. Solution : décider port par port, ce qui rend la remise à null systématique inutile.

```
for q := range fifos {
    if len(fifos[q]) > 0 {
        messOut[q] = <-fifos[q]              // une trame en attente : on l'envoie
    } else {
        messOut[q] = trameNull               // sinon null explicite
    }
}
```

5.2 Files de capacité finie

Q1. Une écriture `fifos[q] <- t` sur une file pleine **bloque** le commutateur. Comme la file ne se vide que plus loin dans la même boucle (extraction + communication), le commutateur ne s'en sort jamais : blocage de tout le réseau synchrone.

Q2. Il faut tester le remplissage avant d'écrire (`len(f) == cap(f)`), jamais écrire dans une file pleine.

```
// Variante 1 : on élimine le message le plus ancien (tête de file)
func ecrire1(f chan trame, t trame) {
    if len(f) == cap(f) { <-f }       // supprime le plus ancien
    f <- t                            // la nouvelle trame prend sa place
}

// Variante 2 : on élimine le nouveau message (file pleine = trame perdue)
func ecrire2(f chan trame, t trame) {
    if len(f) < cap(f) { f <- t }     // sinon : on jette t
}
// Utilisation : remplacer fifos[q] <- t par ecrire1(fifos[q], t) (ou ecrire2).
```

Variante 1 favorise les trames récentes (utile si les anciennes sont périmées). Variante 2 préserve l'ordre et les trames déjà acceptées. Les deux sont sûres car ce code est le seul à écrire dans la file (pas de concurrence).

PARTIE B - Travaux pratiques

TP2 - Élection de leader sur anneau synchrone

Anneau unidirectionnel de `nbNodes = 4` nœuds : le nœud i reçoit sur `tabCan[i]` et émet sur `tabCan[(i+1) % nbNodes]` (le modulo ferme l'anneau : la sortie du nœud 3 est `tabCan[0]`, l'entrée du nœud 0).

2.2 Ossature : main et node

Point clé : les canaux sont synchrones. Si chaque nœud faisait d'abord `out <- m` puis `<-in`, tous seraient bloqués sur leur envoi : interblocage. On envoie donc dans une goroutine pendant que le nœud reçoit, et on attend la fin de l'envoi (fonction `echange`) : c'est le modèle synchrone (tout le monde envoie et reçoit à chaque tour).

```
package main

import "fmt"

const nbNodes int = 4
const nbTours int = 4
const null int = -1

// phase d'échange : envoi et réception simultanés
func echange(in, out chan int, m int) int {
    fin := make(chan bool)
    go func() { out <- m; fin <- true }()   // envoi en parallèle
    recu := <-in                            // réception
    <-fin                                   // l'envoi est terminé aussi
    return recu
}

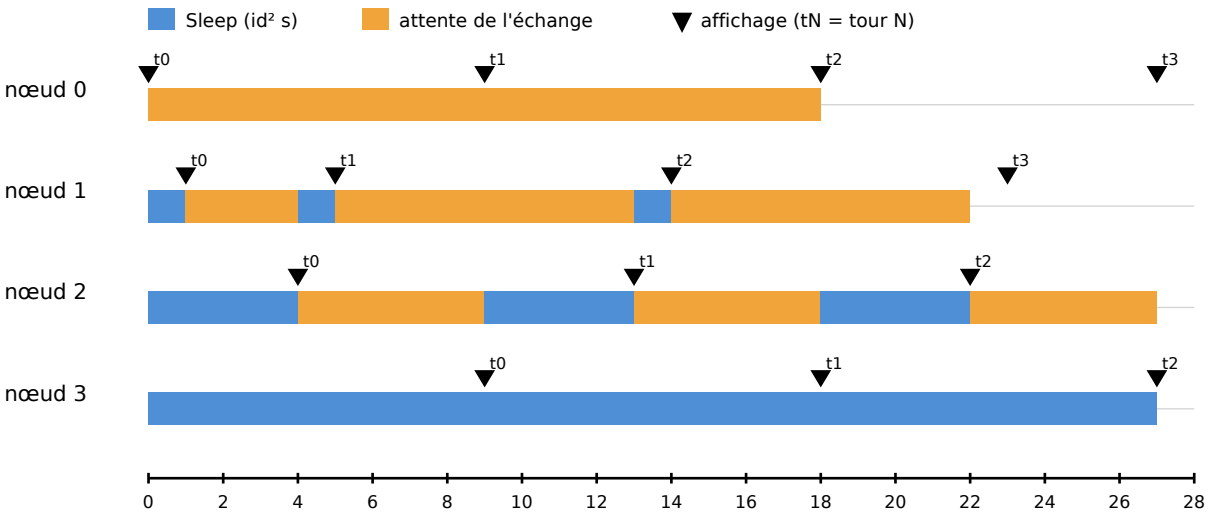
func node(id int, in, out chan int) {
    for tour := 0; tour < nbTours; tour++ {
        recu := echange(in, out, null)      // ici on n'envoie que des null
        fmt.Println("noeud", id, "tour", tour, "recu", recu)
    }
}

func main() {
    var tabCan [nbNodes]chan int
    for i := range tabCan { tabCan[i] = make(chan int) }   // (a) allocation
    fini := make(chan bool)                               // un seul canal de fin
    for i := 0; i < nbNodes; i++ {                        // (b) lancement
        go func(i int) {
            node(i, tabCan[i], tabCan[(i+1)%nbNodes])
            fini <- true
        }(i)
    }
    for i := 0; i < nbNodes; i++ { <-fini }               // (c) attente de la fin
}
```

Test 2.3 : le programme se termine (pas d'interblocage) et chaque nœud affiche 4 lignes (tours 0 à 3) avec `recu -1`. Les nœuds avancent ensemble tour par tour, car chaque échange attend les voisins.

2.4 Temps d'attente et chronogramme

On ajoute `time.Sleep(time.Duration(id*id) * time.Second)` entre l'échange et l'affichage : attentes de 0, 1, 4 et 9 s pour les nœuds 0 à 3. Un échange n'a lieu que quand le nœud **et ses deux voisins** sont prêts. Règle de calcul : l'échange sur un lien i vers i+1 se termine à max(instant où i est prêt, instant où i+1 est prêt) ; le nœud i a fini son tour quand ses deux échanges (envoi et réception) sont finis.



Tour	nœud 0	nœud 1	nœud 2	nœud 3
Tour 0 : échange (tous prêts à t=0), puis affichage à	t=0	t=1	t=4	t=9
Tour 1 : échange terminé à (après attente des voisins)	9	4	9	9
Tour 1 : affichage à	9	5	13	18
Tour 2 : échange terminé à	18	13	18	18
Tour 2 : affichage à	18	14	22	27

Ces valeurs sont celles du chronogramme fourni (0, 1, 4, 9 puis 9, 5, 13, 18, puis 18, 14, 22, 27). À partir du tour 3 : 27, 23, 31, 36.

- **(a)** Oui : après le 1er tour, le nœud 1 est toujours « en avance » (il affiche le tour 1 à t=5, avant le nœud 0 à t=9).
- **(b)** Non : ce n'est pas le nœud 0 (attente nulle), mais le nœud 1. Le nœud 0 reçoit du nœud 3, le plus lent (9 s) : il doit l'attendre. Le nœud 1, lui, n'a que le nœud 0 (déjà prêt) et le nœud 2 (prêt à t=4) à attendre. L'avance dépend de la position dans l'anneau (qui est le voisin lent), pas seulement de sa propre attente.
- **(c)** Non : un nœud ne peut avoir plus d'un tour d'avance sur ses voisins. Pour commencer le tour t+1 il faut échanger avec eux, donc ils doivent avoir terminé le tour t. Les canaux synchrones imposent cette limite.

3 Circulation d'un jeton

```
const jeton int = -2
const idJeton int = 1

func node(id int, in, out chan int, aLeJeton bool) {
    messOut := null
    if aLeJeton { messOut = jeton }              // le détenteur l'envoie au 1er tour
    for tour := 0; tour < nbTours; tour++ {
        messIn := echange(in, out, messOut)
        messOut = null
        if messIn == jeton {
            fmt.Println("noeud", id, "tour", tour, ": reçoit le jeton")
            messOut = jeton                      // reçu au tour i, renvoyé au tour i+1
        }
    }
}
// main : node(i, tabCan[i], tabCan[(i+1)%nbNodes], i == idJeton)
```

Trace attendue (idJeton = 1) : nœud 2 au tour 0, nœud 3 au tour 1, nœud 0 au tour 2, nœud 1 au tour 3.

4 Élection dans un réseau de taille connue

4.1 Numéros uniques

```
var numElec = [nbNodes]int{12, 7, 25, 3}          // uniques, >= 0, distincts des id
// lancement : node(i, numElec[i], tabCan[i], tabCan[(i+1)%nbNodes], i == idJeton)
```

4.2 Phase 1 : élection

Chacun envoie son numéro. Un numéro plus petit que le mien est supprimé (remplacé par null), un plus grand est relayé, le mien qui revient prouve que je suis le maximum. Le numéro du leader fait un tour complet en `nbNodes` tours (émis au tour 0, de retour au tour nbNodes-1) : la boucle fait donc `nbNodes` tours.

```
func node(id, num int, in, out chan int) {
    estLeader := false
    messOut := num                                  // 1er message : mon numéro
    for tour := 0; tour < nbNodes; tour++ {
        messIn := echange(in, out, messOut)
        switch {
        case messIn == null:    messOut = null
        case messIn == num:     estLeader = true; messOut = null   // mon numéro est revenu
        case messIn < num:      messOut = null    // plus petit : je le supprime
        default:                messOut = messIn  // plus grand : je le relaie
        }
        fmt.Println("noeud", id, "tour", tour, "recu", messIn, "leader:", estLeader)
    }
```

L'ordre des cas du `switch` compte : on traite `null` en premier (-1 est inférieur à tout numéro, mais il ne faut pas le confondre avec un numéro plus petit, même si le résultat est le même).

4.3 Phase 2 : diffusion du résultat (boucle suivante, dans la même fonction)

```
    idLeader := -1
    messOut = null
    if estLeader { idLeader = id; messOut = id }  // le leader diffuse son identifiant
    for tour := 0; tour < nbNodes-1; tour++ {
        messIn := echange(in, out, messOut)
        messOut = null
        if messIn != null && !estLeader {
            idLeader = messIn
            messOut = messIn                      // transmis de proche en proche
        }
        fmt.Println("noeud", id, "phase 2 tour", tour, "leader connu:", idLeader)
    }
}
```

Le dernier nœud (à nbNodes-1 sauts) reçoit l'identifiant au tour nbNodes-2, d'où `nbNodes-1` tours (nbNodes convient aussi). Tous les nœuds doivent faire le même nombre de tours, sinon un voisin reste bloqué dans un échange.

5 Élection dans un réseau de taille inconnue

Une seule boucle, infinie en première approche, et des messages à 2 parties : valeur + booléen `estId` (valeur = numéro d'élection ou identifiant du leader). Pour la terminaison automatique simultanée, le message du leader porte aussi le **numéro du tour de fin** (équivalent à un compte à rebours : `fin - tour` tours restants).

```
type message struct {
    val   int
    estId bool      // vrai : val est l'identifiant du leader (phase 2)
    fin   int       // numéro du dernier tour, fixé par le leader
}
var messNull = message{val: null}

func node(id, num int, in, out chan message) {
    estLeader, idLeader, tourFin := false, -1, -1
    messOut := message{val: num}
    for tour := 0; ; tour++ {
        messIn := echange(in, out, messOut)       // echange adapté au type message
        messOut = messNull
        switch {
        case messIn.val == null:
            // rien à faire
        case messIn.estId:                        // phase 2 : identifiant du leader
            idLeader, tourFin = messIn.val, messIn.fin
            if !estLeader { messOut = messIn }    // le leader ne renvoie pas son message
        case messIn.val == num:                   // phase 1 : mon numéro est revenu
            estLeader, idLeader = true, id
            n := tour + 1                         // taille de l'anneau = tours écoulés
            tourFin = 2*n - 1                     // retour du message du leader
            messOut = message{val: id, estId: true, fin: tourFin}
        case messIn.val > num:
```

```
            messOut = messIn                    // plus petit : on laisse messNull
        }
        fmt.Println("noeud", id, "tour", tour, "leader:", idLeader)
        if tour == tourFin { break }            // arrêt simultané partout
    }
}
```

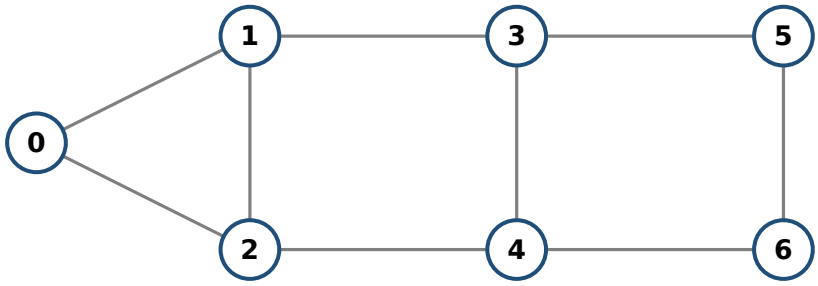
Pourquoi l'arrêt est simultané : le leader se reconnaît au tour n-1, donc n = tour+1 est la taille de l'anneau. Il émet son identifiant au tour n ; le nœud à k sauts le reçoit au tour n+k-1, et le leader le récupère au tour 2n-1. Tous connaissent alors le leader, et tous ont reçu `fin = 2n-1` avant ce tour (le dernier au tour 2n-2). Les numéros de tour sont les mêmes partout (le modèle synchrone les aligne), donc tous s'arrêtent à `tour == 2n-1`.

TP3 partie 1 - Topologie, communications

Le sujet demande de dessiner **votre** topologie de 7 nœuds (chaque nœud a au moins 2 voisins). Voici un exemple complet et vérifié, réutilisé pour le TP4 : 9 liens, donc 18 canaux.

2.1 Schéma, numéros de canaux et ports

Chaque trait = 2 canaux (un par sens)



Méthode : (1) dessiner les liens, 2 flèches par lien ; (2) numéroter les nœuds de 0 à 6 ; (3) numéroter les canaux de 0 à 17 (canal 2k = sens u vers v du lien k, canal 2k+1 = sens v vers u, u < v) ; (4) numéroter les ports de chaque nœud à partir de 0, ici par identifiant de voisin croissant. Le port p est le même indice en entrée et en sortie.

Lien	canal u→v	canal v→u
0-1	0 (0→1)	1 (1→0)
0-2	2 (0→2)	3 (2→0)
1-2	4 (1→2)	5 (2→1)
1-3	6 (1→3)	7 (3→1)
2-4	8 (2→4)	9 (4→2)
3-4	10 (3→4)	11 (4→3)
3-5	12 (3→5)	13 (5→3)
4-6	14 (4→6)	15 (6→4)
5-6	16 (5→6)	17 (6→5)

Nœud	Ports (port : voisin)	comIn (canaux v→nœud)	comOut (canaux nœud→v)
0	0:1, 1:2	1, 3	0, 2
1	0:0, 1:2, 2:3	0, 5, 7	1, 4, 6
2	0:0, 1:1, 2:4	2, 4, 9	3, 5, 8
3	0:1, 1:4, 2:5	6, 11, 13	7, 10, 12
4	0:2, 1:3, 2:6	8, 10, 15	9, 11, 14
5	0:3, 1:6	12, 17	13, 16
6	0:4, 1:5	14, 16	15, 17

2.2 et 3 : main (topologie et lancement)

```
package main

import ("fmt"; "sync")

const nbNodes int = 7
const nbChan int = 18

func main() {
    var tabChan [nbChan]chan int
    for i := range tabChan { tabChan[i] = make(chan int) }

    var comIn [nbNodes][]chan int
    var comOut [nbNodes][]chan int
    comIn[0] = []chan int{tabChan[1], tabChan[3]}
    comOut[0] = []chan int{tabChan[0], tabChan[2]}
```

```
    comIn[1] = []chan int{tabChan[0], tabChan[5], tabChan[7]}
    comOut[1] = []chan int{tabChan[1], tabChan[4], tabChan[6]}
    comIn[2] = []chan int{tabChan[2], tabChan[4], tabChan[9]}
    comOut[2] = []chan int{tabChan[3], tabChan[5], tabChan[8]}
    comIn[3] = []chan int{tabChan[6], tabChan[11], tabChan[13]}
    comOut[3] = []chan int{tabChan[7], tabChan[10], tabChan[12]}
    comIn[4] = []chan int{tabChan[8], tabChan[10], tabChan[15]}
    comOut[4] = []chan int{tabChan[9], tabChan[11], tabChan[14]}
    comIn[5] = []chan int{tabChan[12], tabChan[17]}
    comOut[5] = []chan int{tabChan[13], tabChan[16]}
    comIn[6] = []chan int{tabChan[14], tabChan[16]}
    comOut[6] = []chan int{tabChan[15], tabChan[17]}

    var wg sync.WaitGroup
    wg.Add(nbNodes)                         // nbNodes "Done" à attendre
    for i := 0; i < nbNodes; i++ {
        go func(j int) {                    // j copié : évite de capturer i
            node(j, comIn[j], comOut[j])
            wg.Done()
        }(i)
    }
    wg.Wait()                               // main bloqué jusqu'au dernier Done
}
```

Règle de vérification : pour le nœud Y et son port p vers le voisin X, `comIn[Y][p]` est le canal X→Y et `comOut[Y][p]` le canal Y→X. Le canal `comOut[Y][p]` doit donc se retrouver dans `comIn[X][p']` (p' = port de X vers Y).

4 Fonctions de communication

Un envoi ou une réception sur un canal synchrone bloque : faits à la suite, ils provoqueraient un interblocage. On lance donc **une goroutine par canal**, chacune faisant un `Done` quand sa communication est terminée ; `communication` attend le `WaitGroup`. Les `Add` sont faits **avant** de lancer les goroutines (sinon `Wait` pourrait se débloquer trop tôt).

```
func receive(messIn []int, in []chan int, wg *sync.WaitGroup) {
    wg.Add(len(in))                         // 1. nb de goroutines créées
    for i := range in {                     // 2. une goroutine par canal d'entrée
        go func(i int) {
            messIn[i] = <-in[i]
            wg.Done()                       // communication terminée
        }(i)
    }
}

func send(messOut []int, out []chan int, wg *sync.WaitGroup) {
    wg.Add(len(out))
    for i := range out {
        go func(i int) {
            out[i] <- messOut[i]
            wg.Done()
        }(i)
    }
}

func communication(in []chan int, messIn []int, out []chan int, messOut []int) {
    var wg sync.WaitGroup                   // 1. un waitGroup
    send(messOut, out, &wg)                 // 2. envois
    receive(messIn, in, &wg)                // 3. réceptions
    wg.Wait()                               // 4. attente de toutes les communications
}
```

Chaque goroutine n'écrit que dans sa propre case `messIn[i]` : pas de conflit d'écriture. Le paramètre `i` passé à la fonction anonyme évite que toutes les goroutines partagent la même variable de boucle.

5 Découverte de voisinage direct

```
func node(id int, in, out []chan int) {
    messIn := make([]int, len(in))          // taille adaptée au nb de ports
    messOut := make([]int, len(out))
    for i := range messOut { messOut[i] = id }      // j'envoie mon id à chaque voisin
    communication(in, messIn, out, messOut)
    for p := range messIn {
        fmt.Println("noeud", id, ": port", p, "<- voisin", messIn[p])
    }
}
```

Trace attendue (l'ordre des lignes varie, le contenu non) : pour chaque nœud, le port p affiche l'identifiant du voisin de la colonne « Ports » du tableau ci-dessus. Par exemple, nœud 1 : port 0 voisin 0, port 1 voisin 2, port 2 voisin 3. Un affichage différent, ou un blocage, indique une erreur dans la table des canaux (mauvais indice ou ordre des ports).

TP3 partie 2 - Routage par inondation

1 Nouveau type de message

```
type packet struct {
    from int     // émetteur ; -1 pour un paquet null
    to   int     // destinataire
    num  int     // date d'envoi : tour d'émission
}
var nullPacket = packet{from: -1, to: -1, num: -1}
```

Il suffit ensuite de remplacer `chan int` par `chan packet`, et `[]int` par `[]packet` dans le `main`, dans `communication`, `send` et `receive`.

2.1 et 2.2 Création du paquet et inondation de base

```
func node(id int, in, out []chan packet, emetteur bool, dest int) {
    messIn := make([]packet, len(in))
    messOut := make([]packet, len(out))
    for i := range messOut { messOut[i] = nullPacket }
    if emetteur {                                          // 2.1 paquet initial
        for i := range messOut { messOut[i] = packet{from: id, to: dest, num: 0} }
    }
    for tour := 0; ; tour++ {                              // nombre de tours infini
        communication(in, messIn, out, messOut)
        for i := range messOut { messOut[i] = nullPacket }  // par défaut : rien
        for p := range messIn {
            pk := messIn[p]
            if pk.from == -1 { continue }                  // paquet null
            if pk.to == id {                               // 2.2 je suis le destinataire
                fmt.Println("tour", tour, ": noeud", id, "reçoit", pk)
            } else {
                for q := range messOut { messOut[q] = pk }  // inondation à tous les voisins
            }
        }
    }
}
// main : node(j, comIn[j], comOut[j], j == 0, 6)   (émetteur 0, destinataire 6)
```

Le programme ne se termine jamais : lancez-le avec une limite, par exemple `timeout 2s ./tp3 > trace.txt`.

2.3 Simulation et questions

Résultats obtenus en simulant sur la topologie d'exemple (émetteur 0, destinataire 6, tours numérotés depuis 0) :

- **Q1 - viable ?** Non. Aucun nœud n'élimine jamais un paquet (sauf le destinataire) et chacun le renvoie à tous ses voisins, y compris celui qui vient de le lui envoyer : il rebondit indéfiniment sur chaque lien. Le trafic sature à jusqu'à 18 messages par tour (un par canal) pour toujours (134 messages en 10 tours dans la simulation), et le destinataire reçoit le même paquet en boucle (tours 2, 3, 3, 4, 4, 5...).
- **Q2 - 1re réception :** au tour 2 pour 0 vers 6.
- **Q3 - lien avec la topologie :** c'est la distance (plus court chemin en sauts) entre émetteur et destinataire : 0→2→4→6 = 3 sauts, donc réception au 3e tour, soit l'indice 2 avec une numérotation qui commence à 0.

2.4 Amélioration : ne pas renvoyer aux voisins qui l'ont envoyé ce tour

```
        } else {
            for q := range messOut {
                if messIn[q].from == -1 { messOut[q] = pk }  // port q n'a pas émis ce tour
            }
        }
```

Un port qui a envoyé un paquet non nul pendant ce tour est exclu du renvoi (si plusieurs identiques arrivent, ils sont tous exclus).

Cela résout-il le problème ? Oui en pratique : plus de ping-pong immédiat, et dans la simulation le trafic s'arrête après le tour 4 (0 vers 6, 18 messages au total). Mais il n'y a aucune mémoire : les vagues ne s'éteignent que parce qu'elles se croisent sur les cycles. Le paquet est encore relayé plusieurs fois

par un même nœud, et le destinataire le reçoit 4 fois (tours 2, 3, 3, 4) alors qu'il n'a que 2 voisins.

2.5 Métrique : nombre de messages non nuls reçus

```
nbRecus := 0                                  // avant la boucle
...
avant := nbRecus                              // au début de chaque tour, après communication
for p := range messIn {
    if messIn[p].from != -1 { nbRecus++ }
    ...
}
if nbRecus != avant {
    fmt.Println("noeud", id, "tour", tour, "messages reçus :", nbRecus)
}
```

3 Inondation avec mémorisation

Un tableau `dernier` (taille nbNodes, indexé par l'émetteur) mémorise la date d'envoi du dernier paquet vu, initialisé à -1. Un paquet de date inférieure ou égale à la date mémorisée n'est pas renvoyé ; sinon on mémorise puis on renvoie (sauf aux ports qui viennent d'émettre). **L'émetteur initial doit mémoriser son propre paquet** (`dernier[id] = 0`), sinon il le renverrait en le recevant. Voici la version finale, avec métrique :

```
func node(id int, in, out []chan packet, emetteur bool, dest int) {
    messIn := make([]packet, len(in))
    messOut := make([]packet, len(out))
    dernier := make([]int, nbNodes)
    for i := range dernier { dernier[i] = -1 }
    for i := range messOut { messOut[i] = nullPacket }
    nbRecus := 0
    if emetteur {
        dernier[id] = 0                                  // il connaît déjà son paquet
        for i := range messOut { messOut[i] = packet{from: id, to: dest, num: 0} }
    }
    for tour := 0; ; tour++ {
        communication(in, messIn, out, messOut)
        avant := nbRecus
        for i := range messOut { messOut[i] = nullPacket }
        for p := range messIn {
            pk := messIn[p]
            if pk.from == -1 { continue }
            nbRecus++
            if pk.to == id {                             // destinataire : ne renvoie pas
                fmt.Println("tour", tour, ": noeud", id, "reçoit", pk)
                continue
            }
            if pk.num > dernier[pk.from] {               // paquet nouveau pour moi
                dernier[pk.from] = pk.num                // je mémorise
                for q := range messOut {
                    if messIn[q].from == -1 { messOut[q] = pk }
                }
            }                                            // sinon : déjà vu, ignoré
        }
        if nbRecus != avant {
            fmt.Println("noeud", id, "tour", tour, "messages reçus :", nbRecus)
        }
    }
}
```

3.2 Simulations et réponses

Deux simulations (une seule création de paquet chacune), numérotation des tours depuis 0 :

	Simulation A : 0 → 6	Simulation B : 5 → 0
Distance émetteur-destinataire	3 sauts	3 sauts
Réceptions chez le destinataire (tours)	2 : tours 2 et 3	2 : tours 2 et 3
Dernier tour avec un message en circulation	tour 3 (4 tours : 0 à 3)	tour 3 (4 tours : 0 à 3)
Total de messages (A)	11 (contre 18 avec 2.4, et 134 en 10 tours en base)	-

Pour confirmer, une 3e simulation 3 → 2 : le destinataire reçoit le paquet 3 fois (tours 1, 1 et 2) et le trafic cesse après le tour 2.

- **Q1 - nombre de réceptions :** égal au **nombre de voisins** (degré) du destinataire : chaque voisin lui transmet le paquet une seule fois. Nœud 6 : degré 2, nœud 0 : degré 2, nœud 2 : degré 3.
- **Q2 - fin de circulation :** le dernier échange non nul a lieu au tour 3 dans A et B (le message circule pendant les tours 0 à 3), au tour 2 dans la simulation 3 → 2.
- **Q3 - identique ?** Non, cela dépend de l'émetteur (A et B donnent 3, la simulation 3 → 2 donne 2).
- **Q4 - lien avec la topologie :** oui : le dernier tour de circulation vaut l'**excentricité de l'émetteur** (distance à son plus lointain nœud) : 0 et 5 ont une excentricité de 3, le nœud 3 de 2. Elle est au plus égale au diamètre du réseau, ici 3.

4 (facultatif) Calcul du diamètre : idée d'algorithme

Hypothèses : n connu, un seul paquet à la fois. On enchaîne des boucles finies de n tours (n est une borne sûre sur le diamètre) :

- **Phase 1 (n sous-phases) :** pour s = 0..n-1, le nœud s inonde un paquet (version avec mémorisation). Chaque nœud x note le tour de première réception, d'où la distance d(s, x) (0 pour x = s). Après n sous-phases, x connaît son excentricité ecc(x) = max sur s de d(s, x), car le graphe est non orienté.
- **Phase 2 (n sous-phases) :** pour s = 0..n-1, le nœud s inonde ecc(s) (on ajoute un champ de données au paquet). Chaque nœud conserve le maximum de ce qu'il reçoit, y compris son propre ecc.
- **Résultat :** à la fin de la phase 2 tous les nœuds possèdent max sur x de ecc(x) = diamètre. Coût : $2n^2$ tours.

TP4 - Routage à vecteur de distance (RIP simplifié)

Chaque nœud garde une table `table[d] = {indVois, cout}` (d = identifiant de destination). À chaque tour : (1) échange des tables avec les voisins ; (2) pour chaque destination, si le coût annoncé par un voisin + 1 est meilleur que le coût connu, on adopte ce voisin ; (3) envoi de sa table au tour suivant. Attention : `indVois` est un **indice de port**, pas un identifiant de nœud.

4.1 Adaptation du code du TP3

```
const infini int = 16                       // « +infini » conventionnel (comme RIP)

type infoDest struct {
    indVois int                             // indice du canal de sortie vers le voisin
    cout    int                             // nb de sauts
}
type tableRoutage [nbNodes]infoDest         // type tableau : copié par valeur

// Canaux : make([]chan tableRoutage, ...), chan tableRoutage dans le main.
// communication/send/receive : remplacer int par tableRoutage.
// Lancement des goroutines : node(i, comIn[i], comOut[i])   (3 paramètres)

func node(id int, in, out []chan tableRoutage) {
    tablesIn := make([]tableRoutage, len(in))
    tablesOut := make([]tableRoutage, len(out))

    var table tableRoutage
    for d := range table { table[d] = infoDest{indVois: -1, cout: infini} }
    table[id] = infoDest{indVois: -1, cout: 0}      // case du nœud lui-même
    fmt.Println("id:", id, "table", table)
```

Un type tableau Go est copié à l'affectation : `tablesOut[i] = table` envoie une photographie de la table. Pas de risque que le voisin lise la table pendant qu'on la modifie.

4.2 Boucle synchrone

```
    for i := range tablesOut { tablesOut[i] = table }  // 3. envoi de ma table (1er tour inclus)

    for tour := 1; ; tour++ {
        communication(in, tablesIn, out, tablesOut)      // 1. échange des tables

        for p := range tablesIn {                        // 2. examen de chaque voisin p
            for d := range table {
                c := tablesIn[p][d].cout + 1             // coût via le voisin p
                if c < table[d].cout {                   // strictement meilleur
                    table[d] = infoDest{indVois: p, cout: c}
                }
            }
        }
        fmt.Println("id:", id, "tour", tour, "table", table)

        for i := range tablesOut { tablesOut[i] = table }    // 3. prochain envoi
    }
}
```

Pas besoin de plafonner à `infini` : un voisin inconnu annonce 16, donc 17, jamais strictement inférieur au 16 initial, et la case reste donc inchangée. Une destination à 1 saut est trouvée au tour 1, à k sauts au tour k : l'information avance d'un saut par tour.

5 Détection de la stabilité

```
    nbMaj := 0                                      // avant la boucle
    ...
        modif := false                              // dans la boucle, avant l'examen
        ...
                if c < table[d].cout {
                    table[d] = infoDest{indVois: p, cout: c}
                    modif = true
                }
        ...
        if modif {                                  // au plus un affichage par tour
            nbMaj++
            fmt.Println("id:", id, "tour", tour, "maj n°", nbMaj, "table", table)
        }
```

- **Q1 :** oui, l'affichage s'arrête : après un certain nombre de tours plus aucune table ne change (les programmes, eux, continuent à échanger sans rien modifier).
- **Q2 :** le nombre de tours avec mise à jour est prévisible : c'est l'**excentricité du nœud** (distance à son plus lointain nœud), car chaque tour découvre les destinations à un saut de plus. Dans notre topologie : 3, 3, 3, 2, 2, 3, 3 pour les nœuds 0 à 6 ; le maximum (3) est le diamètre, et c'est aussi le tour à partir duquel tout est stable.

Tables finales attendues sur la topologie d'exemple

Format (indVois, cout) par destination 0 à 6. En cas d'égalité de coût, c'est l'ordre des ports (voisin de plus petit port d'abord) qui tranche, avec la comparaison stricte du code ci-dessus ; une autre implémentation peut donner un autre choix valable.

Nœud	dest 0	dest 1	dest 2	dest 3	dest 4	dest 5	dest 6
0	(-, 0)	(0, 1)	(1, 1)	(0, 2)	(1, 2)	(0, 3)	(1, 3)
1	(0, 1)	(-, 0)	(1, 1)	(2, 1)	(1, 2)	(2, 2)	(1, 3)
2	(0, 1)	(1, 1)	(-, 0)	(1, 2)	(2, 1)	(1, 3)	(2, 2)
3	(0, 2)	(0, 1)	(0, 2)	(-, 0)	(1, 1)	(2, 1)	(1, 2)
4	(0, 2)	(0, 2)	(0, 1)	(1, 1)	(-, 0)	(1, 2)	(2, 1)
5	(0, 3)	(0, 2)	(0, 3)	(0, 1)	(0, 2)	(-, 0)	(1, 1)
6	(0, 3)	(0, 3)	(0, 2)	(0, 2)	(0, 1)	(1, 1)	(-, 0)